

2주차 예습과제 (pp.85-180)

☼ 상태	Done
👤 담당자	이서윤(인공지능대학 인공지능데이터사이언스학부)
📅 마감일	09/14/2026
📁 작업 유형	

2. 사이킷런으로 시작하는 머신러닝

사이킷런 소개와 특징

사이킷런(scikit-learn) - 파이썬 머신러닝 라이브러리 중 가장 많이 사용되는 라이브러리이며 파이썬 기반의 머신러닝을 위한 가장 쉽고 효율적인 개발 라이브러리를 제공함

사이킷런의 특징

- 파이썬 기반의 다른 머신러닝 패키지도 사이킷런 스타일의 API를 지향할 정도로 쉽고 가장 파이썬스러운 API를 제공한다
- 머신러닝을 위한 매우 다양한 알고리즘과 개발을 위한 편리한 프레임워크와 API를 제공한다
- 오랜 기간 실전 환경에서 검증됐으며, 매우 많은 환경에서 사용되는 성숙한 라이브러리이다

첫 번째 머신러닝 만들어 보기 - 붓꽃 품종 예측하기

붓꽃 데이터 세트로 붓꽃의 품종을 분류(Classification)



지도학습 (Supervised Learning) - 학습을 위한 다양한 피쳐와 분류 결정값인 레이블 (Label) 데이터로 모델을 학습한 뒤, 별도의 테스트 데이터 세트에서 미지의 레이블을 예측

함

학습 데이터 세트로 학습 → 테스트 데이터 세트로 머신러닝 모델의 예측 성능을 평가

- **사이킷런(scikit-learn)**

- 패키지 내 모듈명은 `sklearn` 으로 시작

- `sklearn.datasets`

- 사이킷런에서 자체적으로 제공하는 데이터 세트를 생성하는 모듈의 모임
- 붓꽃 데이터 세트 생성: `load_iris()`

- `sklearn.tree`

- 트리 기반 ML 알고리즘을 구현한 클래스의 모임
- 의사 결정 트리(Decision Tree) 구현: `DecisionTreeClassifier`

- `sklearn.model_selection`

- 데이터를 학습 데이터, 검증 데이터, 예측 데이터로 분리
- 최적의 하이퍼 파라미터로 평가하기 위한 다양한 모듈의 모임

- **하이퍼 파라미터(Hyper Parameter)**

- 머신러닝 알고리즘별로 최적의 학습을 위해 직접 입력하는 파라미터
- 하이퍼 파라미터를 통해 머신러닝 알고리즘의 성능을 튜닝할 수 있음

`train_test_split()`

- 데이터 세트를 학습 데이터와 테스트 데이터로 분리하는 데 사용하는 함수

```
X_train, X_test, y_train, y_test = train_test_split(iris_data, iris_label,
                                                    test_size=0.2, random_state=11)
```

- 주요 파라미터

- `iris_data` → 피처(Feature) 데이터 세트
- `iris_label` → 레이블(Label) 데이터 세트
- `test_size=0.2` → 전체 데이터 세트 중 테스트 데이터 세트의 비율을 20%로 지정
- `random_state` → 호출할 때마다 동일한 학습/테스트 데이터 세트를 생성하기 위한 난수 발생 값
 - random 값을 만드는 seed와 같은 의미

- 숫자 자체는 어떤 값을 지정해도 상관없음
- 지정하지 않으면 수행할 때마다 서로 다른 데이터 세트로 분리될 수 있음

DecisionTreeClassifier

- `DecisionTreeClassifier` 객체 생성 → 학습 수행 → 예측 수행 → 예측 성능 평가
- `accuracy_score()` 함수로 정확도(예측 결과가 실제 레이블 값과 얼마나 정확하게 맞는지 를 평가) 측정

사이킷런의 기반 프레임워크 익히기

Estimator 이해 및 `fit()`, `predict()` 메서드

`fit()`

- 지도학습에서 **ML 모델을 학습시키는 메서드**

`predict()`

- 학습된 모델을 이용해 **예측 결과를 반환하는 메서드**

Estimator 클래스

Classifier

- **분류(Classification) 알고리즘을 구현한 클래스**

Regressor

- **회귀(Regression) 알고리즘을 구현한 클래스**

Estimator

- `Classifier` + `Regressor` 를 통칭
- 지도학습의 모든 알고리즘을 구현한 클래스를 통칭
- 내부에 `fit()` 과 `predict()` 를 구현함

Estimator를 활용하는 기능

`cross_val_score()`

- Estimator를 인자로 받아 `fit()` 과 `predict()` 를 호출하여 모델을 평가

GridSearchCV

- Estimator를 인자로 받아 `fit()` 과 `predict()` 를 이용해 평가 및 하이퍼 파라미터 튜닝 수행

비지도학습의 `fit()` 과 `transform()`

`fit()`

- 지도학습에서의 '모델 학습'과는 의미가 다름
- 입력 데이터의 형태에 맞춰 **데이터를 변환하기 위한 사전 구조를 맞추는 작업**

`transform()`

- `fit()` 으로 사전 구조를 맞춘 뒤 **실제 데이터 변환 작업을 수행**
- 차원 변환, 클러스터링, 피쳐 추출 등에 사용

`fit_transform()`

- `fit()` + `transform()` 을 한 번에 수행
- 두 메서드를 별도로 호출할 필요를 줄여줌
- 사용 시 주의가 필요함

사이킷런의 주요 모듈

분류	모듈명	설명
예제 데이터	sklearn.datasets	사이킷런에 내장되어 예제로 제공하는 데이터 세트
피쳐 처리	sklearn.preprocessing	데이터 전처리에 필요한 다양한 가공 기능 제공
	sklearn.feature_selection	알고리즘에 큰 영향을 미치는 피쳐를 우선순위대로 선택 작업을 수행하는 다양한 기능 제공
	sklearn.feature_extraction	텍스트 데이터나 이미지 데이터의 벡터화된 피쳐를 추출하는 데 사용
피쳐 처리 & 차원 축소	sklearn.decomposition	차원 축소와 관련한 알고리즘을 지원하는 모듈
데이터 분리, 검증 & 파라미터 튜닝	sklearn.model_selection	교차 검증을 위한 학습용/테스트용 분리, 그리드 서치(Grid Search)로 최적 파라미터 추출 등의 API 제공
평가	sklearn.metrics	분류, 회귀, 클러스터링, 페어와이즈(Pairwise)에 대한 다양한 성능 측정 방법 제공
ML 알고리즘	sklearn.ensemble	앙상블 알고리즘 제공 랜덤 포레스트, 에이다 부스트, 그래디언트 부스팅 등을 제공

분류	모듈명	설명
	sklearn.linear_model	선형 회귀, 릿지(Ridge), 라쏘(Lasso) 및 로지스틱 회귀 등 회귀 관련 알고리즘을 지원 SGD(Stochastic Gradient Descent) 관련 알고리즘 제공
	sklearn.naive_bayes	나이브 베이즈 알고리즘 제공
	sklearn.neighbors	최근접 이웃 알고리즘 제공
	sklearn.svm	서포트 벡터 머신 알고리즘 제공
	sklearn.tree	의사 결정 트리 알고리즘 제공
	sklearn.cluster	비지도 클러스터링 알고리즘 제공
유틸리티	sklearn.pipeline	피처 처리 등의 변환과 ML 알고리즘 학습, 예측 등을 함께 묶어서 실행할 수 있는 유틸리티 제공

내장된 예제 데이터 세트

사이킷런의 데이터 세트는 datasets 모듈에 있는 여러 API를 호출해 만들 수 있음

분류나 회귀를 연습하기 위한 예제용도의 데이터 세트 / 분류나 클러스터링을 위해 표본 데이터로 생성될 수 있는 데이터 세트로 나뉘어짐

fetch 계열의 명령

- 데이터의 크기가 커서 패키지에 처음부터 저장돼 있지 않고 인터넷에서 내려받아 홈 디렉터리 아래의 `scikit_learn_data` 라는 서브 디렉터리에 저장한 후 추후 불러들이는 데이터

표본 데이터 생성기

API 명	설명
<code>datasets.make_classification()</code>	분류를 위한 데이터 세트를 만듭니다. 특히 높은 상관도, 불필요한 속성 등의 노이즈 효과를 위한 데이터를 무작위로 생성해 줍니다.
<code>datasets.make_blobs()</code>	클러스터링을 위한 데이터 세트를 무작위로 생성해 줍니다. 군집 지정 개수에 따라 여러 가지 클러스터링을 위한 데이터 세트를 쉽게 만들어 줍니다.

Key

- 보통 data, target, target_name, feature_names, DESCR로 구성돼 있음
- data, target - 넘파이 배열(ndarray) 타입 / target_names, feature_names - 넘파이 배열 또는 파이썬 리스트(list) 타입 / DESCR - 스트링 타입
- 피처의 데이터 값을 반환받기 위해서는 내장 데이터 세트 API를 호출한 뒤에 그 Key값을 지정하면 된다

ipython 코드 실습

Model Selection 모듈 소개

사이킷런의 model_selection 모듈은 학습 데이터와 테스트 데이터 세트를 분리하거나 교차 검증 분

할 및 평가, 그리고 Estimator의 하이퍼 파라미터를 튜닝하기 위한 다양한 함수와 클래스를 제공함

학습/테스트 데이터 세트 분리 - `train_test_split()`

- 예측을 수행하는 데이터 세트는 학습을 수행한 학습용 데이터 세트가 아닌 전용의 테스트 데이터 세트여야 함

`train_test_split()`

- `sklearn.model_selection` 모듈에서 제공
- 전체 데이터 세트를 **학습용 데이터와 테스트용 데이터로 분리**
- 첫 번째 파라미터 → 피처 데이터 세트
- 두 번째 파라미터 → 레이블 데이터 세트

주요 파라미터

`test_size`

- 전체 데이터에서 **테스트 데이터 세트의 크기를 얼마로 샘플링할지 결정**
- 디폴트: **0.25 (25%)**

`train_size`

- 전체 데이터에서 **학습용 데이터 세트의 크기를 얼마로 샘플링할지 결정**
- 일반적으로 `test_size` 를 사용하므로 **잘 사용되지 않음**

`shuffle`

- 데이터를 분리하기 전에 **미리 섞을지 여부를 결정**
- 디폴트: **True**

- 데이터를 분산시켜 좀 더 효율적인 학습 및 테스트 데이터 세트 생성

`random_state`

- 호출할 때마다 동일한 학습/테스트용 데이터 세트를 생성하기 위한 난수 값
- 지정하지 않으면 무작위로 분리되어 수행할 때마다 다른 데이터 세트가 생성될 수 있음

반환값

- `train_test_split()` 은 튜플(Tuple) 형태로 다음 순서로 반환함
- `X_train` → `X_test` → `y_train` → `y_test`

학습을 위한 데이터의 양을 일정 수준 이상으로 보장하는 것도 중요하지만, 학습된 모델에 대해 다양한 데이터를 기반으로 예측 성능을 평가해보는 것도 매우 중요함

교차 검증

과적합(Overfitting)

- 모델이 학습 데이터에만 과도하게 최적화되어 새로운 데이터에서는 예측 성능이 떨어지는 현상
- 고정된 테스트 데이터만 반복 평가하면 테스트 데이터에도 과적합될 수 있음

교차 검증(Cross Validation)

- 데이터 편종을 막기 위해 여러 학습 데이터 세트와 검증 데이터 세트에서 반복적으로 학습·평가하는 방법
- 다양한 데이터로 평가하여 모델의 성능을 보다 신뢰성 있게 검증
- 평가 결과를 바탕으로 하이퍼 파라미터 튜닝 등 모델 최적화 가능

데이터 세트 구분

- 학습(Train) 데이터 → 모델 학습
- 검증(Validation) 데이터 → 학습된 모델을 다양하게 평가하고 최적화
- 테스트(Test) 데이터 → 최종 모델의 예측 성능 평가

K 폴드 교차 검증

먼저 K개의 데이터 폴드 세트를 만들어서 K번만큼 각 폴드 세트에 학습과 검증 평가를 반복적으로 수행하는 방법



사이킷런에서는 K 폴드 교차 검증 프로세스를 구현하기 위해 KFold와 StratifiedKFold 클래스를 제공함

[ipynb 코드 실습](#)

Stratified K 폴드

- 불균형한(imbalanced) 분포도를 가진 레이블(결정 클래스) 데이터 집합을 위한 K 폴드 방식
- 불균형한 분포도를 가진 레이블 데이터 집합 - 특정 레이블 값이 특이하게 많거나 매우 적어서 값의 분포가 한쪽으로 치우치는 것
- 원본 데이터의 레이블 분포를 먼저 고려한 뒤 이 분포와 동일하게 학습과 검증 데이터 세트를 분배함
- 왜곡된 레이블 데이터 세트에서는 반드시 **Stratified K 폴드**를 이용해 교차 검증해야 함
- 분류(Classification)에서의 교차 검증은 K 폴드가 아니라 **Stratified K 폴드**로 분할되어야 하며 회귀(Regression)에서는 **Stratified K 폴드**가 지원되지 않음

[ipynb 코드 실습](#)

교차 검증을 보다 간편하게 — `cross_val_score()`

`cross_val_score()`

- 사이킷런에서 교차 검증을 한꺼번에 수행해주는 API
- K 폴드에서 직접 반복했던 데이터 분할 → 학습(`fit`) → 예측(`predict`) → 평가 과정을 자동 수행

주요 파라미터

- `estimator` → 사용할 ML 알고리즘
- `X` → 피쳐 데이터 세트
- `y` → 레이블 데이터 세트
- `scoring` → 예측 성능 평가 지표
- `cv` → 교차 검증 폴드(Fold) 수

동작 및 반환값

- `cv`에 지정된 횟수만큼 교차 검증을 수행하며 각 폴드에서 `scoring`으로 지정한 평가 지표 측정값을 배열 형태로 반환함
- 일반적으로 반환된 값들의 평균을 최종 평가 수치로 사용함

`cross_validate()` 와 차이

- `cross_val_score()` → 하나의 평가 지표 사용
- `cross_validate()` → 여러 평가 지표 사용 가능 + 학습 데이터 성능 및 수행 시간 제공

ipynb 코드 실습

GridSearchCV - 교차 검증과 최적 하이퍼 파라미터 튜닝을 한 번에

- 사이킷런은 **GridSearchCV** API를 이용해 Classifier나 Regressor와 같은 알고리즘에 사용되는 하이퍼 파라미터를 순차적으로 입력하면서 편리하게 최적의 파라미터를 도출할 수 있는 방안을 제공함
- **GridSearchCV**는 교차 검증을 기반으로 최적 하이퍼 파라미터 값을 찾음
 - 데이터 세트를 cross-validation을 위한 학습/테스트 세트로 자동으로 분할한 뒤에 하이퍼 파라미터 그리드에 기술된 모든 파라미터를 순차적으로 적용해 최적의 파라미터를 찾을 수 있게 해줌

주요 파라미터

- `estimator`
 - 하이퍼 파라미터를 튜닝할 ML 알고리즘 객체
 - `Classifier`, `Regressor`, `Pipeline` 사용 가능
- `param_grid`
 - 튜닝할 하이퍼 파라미터명과 여러 파라미터 값을 딕셔너리 형태로 지정

- **key** → 파라미터명
- **value** → 적용할 여러 파라미터 값의 리스트
- **scoring**
 - 예측 성능을 측정할 평가 방법 지정
 - 별도의 성능 평가 함수도 지정 가능
- **cv**
 - 교차 검증을 위해 분할할 학습/테스트 세트의 개수 지정
- **refit**
 - 디폴트: **True**
 - 최적의 하이퍼 파라미터를 찾은 뒤, 해당 파라미터로 estimator를 다시 학습

ipynb 코드 실습

데이터 전처리 (Data Preprocessing)

ML 알고리즘은 데이터에 기반하고 있기 때문에 어떤 데이터를 입력으로 가지느냐에 따라 결과도 크게 달라질 수 있다

결손값(Null/NaN) 처리

- 사이킷런 ML 알고리즘은 Null 값을 허용하지 않음
- 따라서 다른 값으로 대체하거나 피처를 삭제해야 함
 - Null이 적음 → 평균값 등으로 대체
 - Null이 대부분 → 해당 피처 드롭(Drop)
 - 중요한 피처 → 업무 로직 등을 검토하여 정밀한 대체 값 선정

문자열 값 처리

- 사이킷런 ML 알고리즘은 문자열 값을 입력값으로 허용하지 않음
- 따라서 문자열을 숫자형으로 변환(인코딩)해야 함
 - 카테고리형 피처 → 숫자형 코드 값으로 인코딩
 - 텍스트형 피처 → 피처 벡터화(Feature Vectorization) 수행
 - 불필요한 문자열 피처 → 삭제
- 식별자 피처(주민번호, 단순 문자열 ID 등)
 - 예측에 중요한 요소가 아니므로 삭제하는 것이 좋음

데이터 인코딩

레이블 인코딩(Label encoding)

- 카테고리 피처를 코드형 숫자 값으로 변환
 - 예: TV → 1, 냉장고 → 2, 컴퓨터 → 3
- '01', '02' 같은 코드도 문자열이므로 숫자형으로 변환 필요
- 사이킷런의 `LabelEncoder` 사용

단점

- 숫자의 크고 작음이 순서나 중요도로 잘못 인식될 수 있음
- 실제로는 단순 코드일 뿐인데 ML 알고리즘이 $1 < 2 < 3$ 의 의미가 있다고 판단할 수 있음
- 따라서 선형 회귀 등에는 적용하지 않는 것이 좋으며 트리 계열 ML 알고리즘에서는 이러한 특성을 반영하지 않으므로 사용 가능

원-핫 인코딩(One-Hot Encoding)

- 레이블 인코딩의 숫자 크기 문제를 해결하는 방식
- 카테고리의 고유 값마다 새로운 피처(칼럼)를 생성하며 해당 값의 칼럼만 1, 나머지는 0으로 표시
 - 예: TV → TV=1, 냉장고=0, 컴퓨터=0
- 사이킷런의 `OneHotEncoder` 사용
 - 입력값은 2차원 데이터 필요
 - 결과는 희소 행렬(Sparse Matrix) → `toarray()` 로 밀집 행렬(Dense Matrix) 변환 가능

판다스 `get_dummies()`

- 판다스에서 원-핫 인코딩을 간편하게 수행하는 API
- 문자열 카테고리 값을 별도로 숫자형으로 변환하지 않고 바로 원-핫 인코딩 가능

피처 스케일링과 정규화

표준화 (Standardization)

데이터의 피처 각각이 평균이 0이고 분산이 1인 가우시안 정규 분포를 가진 값으로 변환하는 것

$$x_{i_new} = \frac{x_i - \text{mean}(x)}{\text{stdev}(x)}$$

정규화 (Normalization)

변수를 모두 동일한 크기 단위로 비교하기 위해 값을 모두 최소 0 ~ 최대 1의 값으로 변환하는 것

$$x_{i_new} = \frac{x_i - \min(x)}{\max(x) - \min(x)}$$

Normalizer 모듈

사이킷런의 **Normalizer 모듈**은 선형대수에서의 정규화 개념이 적용됐으며, 개별 벡터의 크기를 맞추기 위해 변환하는 것을 의미한다

$$x_{i_new} = \frac{x_i}{\sqrt{x_i^2 + y_i^2 + z_i^2}}$$

StandardScaler

표준화를 쉽게 지원하기 위한 클래스이며 개별 피처를 평균이 0이고 분산이 1인 값으로 변환해준다

[ipynb 코드 실습](#)

MinMaxScaler

데이터값을 0과 1사이의 범위 값으로 변환하며(음수 값이 있으면 -1 에서 1값으로 변환) 데이터의 분포가 가우시안 분포가 아닐 경우에 Min, Max Scale을 적용해 볼 수 있다

[ipynb 코드 실습](#)

학습 데이터와 테스트 데이터의 스케일링 변환 시 유의점

스케일링 시 `fit()` / `transform()` 주의사항

학습 데이터

- `fit()` → `transform()` 또는 `fit_transform()` 사용

테스트 데이터

- 다시 `fit()` 하면 안 됨
- 학습 데이터로 이미 `fit()` 한 **Scaler 객체의 기준을 그대로 사용하여 `transform()` 만 수행**
- 테스트 데이터에 `fit_transform()` 도 사용하면 안됨

이유: 학습/테스트 데이터의 스케일링 기준이 서로 달라지면 올바른 예측 결과를 얻기 어려움

3. 평가

분류의 성능 평가 지표

- 정확도(Accuracy)
- 오차행렬(Confusion Matrix)
- 정밀도(Precision)
- 재현율(Recall)
- F1 스코어
- ROC AUC

이진/멀티 분류 모두에 적용되는 지표이지만, 특히 **이진 분류**에서 더욱 중요하게 강조하는 지표이다

정확도(Accuracy)

실제 데이터에서 예측 데이터가 얼마나 같은지를 판단하는 지표

$$\text{정확도(Accuracy)} = \frac{\text{예측 결과가 동일한 데이터 건수}}{\text{전체 예측 데이터 건수}}$$

불균형한 레이블 데이터 세트에서는 성능 수치로 사용해서는 안됨

[ipynb 코드 실습](#)

오차 행렬

학습된 분류 모델이 예측을 수행하면서 얼마나 헛갈리고(confused) 있는지도 함께 보여주는 지표

이진 분류의 예측 오류가 얼마인지와 더불어 어떠한 유형의 예측 오류가 발생하고 있는지를 함께 나타내는 지표

		예측 클래스 (Predicted Class)	
		Negative(0)	Positive(1)
실제 클래스 (Actual Class)	Negative(0)	TN (True Negative)	FP (False Positive)
	Positive(1)	FN (False Negative)	TP (True Positive)

4분면 행렬에서 실제 레이블 클래스 값과 예측 레이블 클래스 값이 어떠한 유형을 가지고 매핑되는지를 나타냄

- TN는 예측값을 Negative 값 0으로 예측했고 실제 값 역시 Negative 값 0
- FP는 예측값을 Positive 값 1로 예측했는데 실제 값은 Negative 값 0
- FN은 예측값을 Negative값 0으로 예측했는데 실제 값은 Positive 값 1
- TP는 예측값을 Positive값 1로 예측했는데 실제 값 역시 Positive 값 1

[ipynb 코드 실습](#)

정밀도와 재현율

정밀도(Precision)

- Positive로 예측한 데이터 중 실제로 Positive인 비율
- **정밀도 = $TP / (FP + TP)$**
- 양성 예측도라고도 함
- FP를 낮추는 것이 중요

재현율(Recall)

- 실제 Positive 데이터 중 Positive로 정확하게 예측한 비율
- **재현율 = $TP / (FN + TP)$**
- 민감도(Sensitivity), TPR(True Positive Rate)이라고도 함
- FN을 낮추는 것이 중요

어떤 지표가 더 중요한가?

재현율이 중요한 경우

- 실제 Positive를 Negative로 잘못 판단(FN)했을 때 손실이 큰 경우
- 예: 암 진단, 금융 사기 적발
- → 실제 암 환자/사기 거래를 놓치지 않는 것이 중요

정밀도가 중요한 경우

- 실제 Negative를 Positive로 잘못 판단(FP)했을 때 손실이 큰 경우
- 예: 스팸 메일 분류
- → 정상 메일을 스팸으로 잘못 분류하지 않는 것이 중요

ipy nb 코드 실습

정밀도/재현율 트레이드오프

분류 결정 임계값(Threshold)

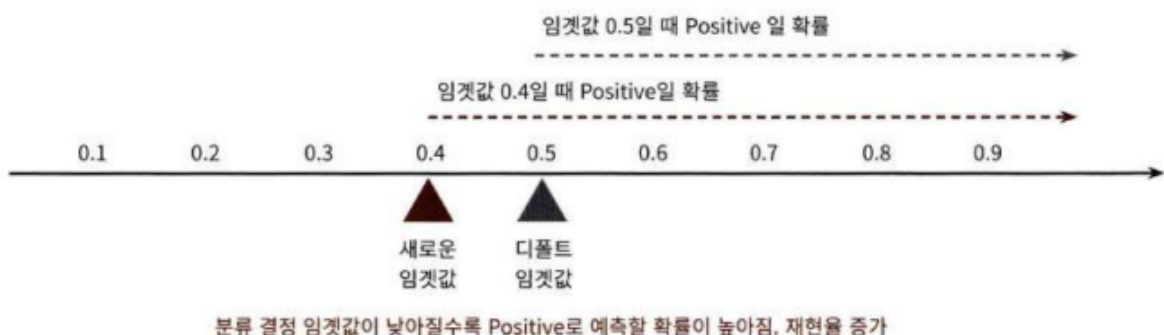
- 예측 확률을 기준으로 Positive / Negative를 결정하는 기준값
- 일반적인 이진 분류의 임계값은 0.5(50%)
- 예: Positive 확률이 0.5보다 크면 → Positive(1)로 예측

정밀도/재현율 트레이드오프

- 임계값을 조정하여 정밀도 또는 재현율을 높일 수 있음
- 하지만 두 지표는 상호 보완적이므로 한쪽을 높이면 다른 쪽이 떨어지기 쉬움

임계값 ↓ → Positive 예측 증가 → 재현율↑ / 정밀도↓

임계값 ↑ → Positive 예측 감소 → 정밀도↑ / 재현율↓



`predict_proba()`

- 학습된 **Classifier**에서 개별 클래스의 예측 확률을 반환하는 메서드
- 입력 → 테스트 피쳐 데이터 세트, 반환 → 클래스별 예측 확률을 ndarray 형태로 반환

이진 분류의 경우 : [0 클래스(Negative) 확률, 1 클래스(Positive) 확률]

- 예: [0.1, 0.9] → Negative 10%, Positive 90% → 1로 예측
- 데이터 100개 + 클래스 2개 → 100 × 2 ndarray 반환

`predict()` 와 차이

- `predict_proba()` → 클래스별 예측 확률 반환
- `predict()` → 예측 확률과 결정 임계값을 기준으로 최종 클래스값 반환

정밀도와 재현율의 맹점

Positive 예측의 임계값을 변경함에 따라 정밀도와 재현율의 수치가 변경된다

임계값의 이러한 변경은 업무 환경에 맞게 두 개의 수치를 상호 보완할 수 있는 수준에서 적용되어야 하며 그렇지 않고 단순히 하나의 성능 지표 수치를 높이기 위한 수단으로 사용해서는 안된다

정밀도 또는 재현율 중 하나만 스코어가 좋고 다른 하나는 스코어가 나쁜 분류는 성능이 좋지 않은 분류로 간주할 수 있음

F1 스코어

정밀도와 재현율을 결합한 지표이며 정밀도와 재현율이 어느 한 쪽으로 치우치지 않는 수치를 나타낼 때 상대적으로 높은 값을 가진다

$$F1 = \frac{2}{\frac{1}{recall} + \frac{1}{precision}} = 2 * \frac{precision * recall}{precision + recall}$$

[ipynb 코드 실습](#)

ROC 곡선과 AUC

ROC 곡선 (Receiver Operation Characteristic Curve) - FPR(False Positive Rate)

이 변할 때 TPR(True

Positive Rate)이 어떻게 변하는지를 나타내는 곡선

TPR(True Positive Rate) = 재현율 = 민감도(Sensitivity)

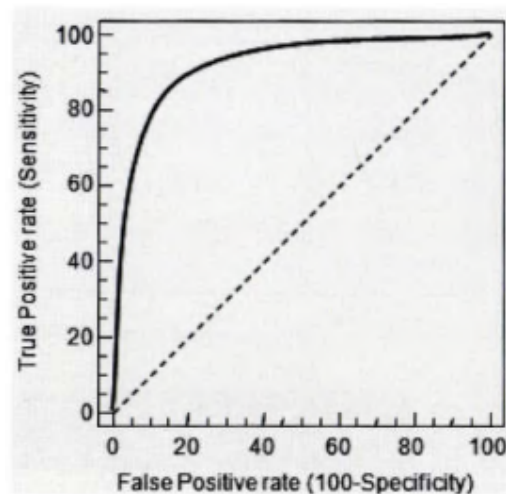
- 실제 Positive 중 **Positive**로 정확히 예측한 비율
- `TPR = TP / (FN + TP)`

TNR(True Negative Rate) = 특이성(Specificity)

- 실제 Negative 중 **Negative**로 정확히 예측한 비율
- $TNR = TN / (FP + TN)$

FPR(False Positive Rate)

- 실제 Negative 중 **Positive**로 잘못 예측한 비율
- $FPR = FP / (FP + TN)$
- $FPR = 1 - TNR = 1 - \text{특이성}$



〈ROC 곡선 예시〉

분류 결정 임계값(Threshold)을 변화시키면서 FPR과 TPR의 변화를 나타낸 곡선

임계값 = 1

→ 거의 모두 Negative로 예측

→ $FP = 0$ → $FPR = 0$

임계값 = 0

→ 거의 모두 Positive로 예측

→ $TN = 0$ → $FPR = 1$

즉, 임계값을 1 → 0으로 변화시키면서 FPR과 TPR을 계산해 ROC 곡선을 생성함

ROC 성능 해석

- 대각선 수준 → 랜덤 분류 수준, $AUC = 0.5$
- ROC 곡선이 대각선에서 멀어지고 **왼쪽 위에 가까울수록 성능이 좋음**
- 사이킷런에서는 `roc_curve()` API를 이용해 ROC 곡선을 구할 수 있음

ipynb 코드 실습